

Databases

Homework 4: Full-Text Search Using MySQL for Your Domain

Guidelines

Author: Vadim Ermolayev

Introduction	2
1 Steps to Accomplish.....	2
1.0 Read the Methodological Guideline.....	2
1.1 Create a MyISAM Table to Extend Your Database	2
1.2 Populate Your MyISAM Table with Relevant Textual Data	2
1.3 Alter the Stop Word List if Needed	2
1.4 Create the Full-Text Index	2
1.5 Elaborate Queries IN NATURAL LANGUAGE MODE.....	3
1.6 Elaborate Queries IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION.....	3
1.7 Elaborate Queries IN BOOLEAN MODE	3
1.8 Conclude Your Report	3
2 Full-Text Search Using MySQL	3
2.1 MATCH ... AGAINST Operator.....	4
2.2 Full-Text Search Modes	4
2.2.1 IN NATURAL LANGUAGE MODE.....	4
2.2.2 IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION.....	5
2.2.3 IN BOOLEAN MODE.....	5
2.3 MySQL Server Configuration Changes	7
3 Grading.....	7
References	7

Introduction

This document contains the guidelines for a student to perform their Homework 4. This homework is the last individual homework assignment in the Databases course.

1 Steps to Accomplish

Your homework comprises the accomplishment of the following steps 1.0 to 1.8. Your homework report should contain all the bits of information that are necessary for the reproduction and explicit interpretation of your actions and results. Such bits of information include: the state of your Database before action, the state of your Database after action, explanation of the query and results. These should be provided as screenshots in the corresponding section of your report. More in-context recommendations are given within the descriptions of the Steps below. The report must be structured using the titles of the subsections 1.0 – 1.8 in this section.

Students are allowed to use any other full-text search Database Engine then MyISAM (c.f. Sect 1.1 and further in the document). However, if any alternative is used, the homework report must contain the justification of the choice.

1.0 Read the Methodological Guideline

The guideline on and workflow of how to refine your Database (HW2 & 3) for enabling full-text search in it are given in Sect. 2 below. Please go through the guideline to understand how to perform all the necessary steps to: (i) extend your Database; (ii) populate your Database with a small set of texts to be searched; and (iii) perform several full-text SELECT queries in different modes.

1.1 Create a **MyISAM** Table to Extend Your Database

Please create at least two columns in your table. This table will be used to store textual documentation explaining the semantics of some entities in your Database. Hence, the first text attribute might be for the names of Entities (like e.g. Lufthansa or Boris Johnson); the second attribute might be for storing textual descriptions of the corresponding Entities. E.g. for Boris Johnson, you might wish to use the text from his Wiki page¹. The relationship of the Entities to the domain of your variant and your database shall be explained in the report. Please document the results of this step in the report by providing: (i) the DDL query; (ii) the result of its execution; and (iii) the explanation of the relationship mentioned above.

1.2 Populate Your **MyISAM** Table with Relevant Textual Data

Please store **at least** 20 Entity names and 20 corresponding natural-language texts. Please document your results in the report by providing: (i) the query(ies) used for loading external data into the MyISAM table; (ii) the proofs of the successful executions of these queries; (iii) populated MyISAM table in the Database.

1.3 Alter the Stop Word List if Needed

Please review the built-in Stop Word list² provided by MySQL Server. If you figure out that the built-in list misses some words or needs to be updated in some other way, please create your own Stop Word list and change MySQL server configuration appropriately. Please explain your choice in the report.

1.4 Create the Full-Text Index

Create the full-text index(es) for your additional MyISAM table covering all the text attributes that need to be searched. This could be done at Step 1.1 using the CREATE TABLE statement. If your Stop Word list has been altered in Step 1.3 then you need to re-create the indexes using, e.g. the

¹ https://en.wikipedia.org/wiki/Boris_Johnson

² https://dev.mysql.com/doc/refman/8.4/en/fulltext-Stop_Words.html#fulltext-Stop_Words-Stop_Words-for-mysam-search-indexes

ALTER TABLE statement. Please explain your choice and provide the results of indexes creation, as screenshots, in your report.

1.5 Elaborate Queries IN NATURAL LANGUAGE MODE

Elaborate 3 FULLTEXT search queries IN NATURAL LANGUAGE MODE. Please include more than one word in at least one query. For every query, please report: (i) the query; (ii) the first 20 results with the highest relevance; (iii) your explanations about the correctness of the results.

1.6 Elaborate Queries IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION

Elaborate 3 FULLTEXT search queries IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION. Please use the same search strings as at the previous Step 1.5. For every query, please report: (i) the query; (ii) the first 20 results with the highest relevance; (iii) your explanations about the correctness and utility of the results using the comparison to the result of the similar query of Step 1.5.

1.7 Elaborate Queries IN BOOLEAN MODE

Elaborate 6 FULLTEXT search queries IN BOOLEAN MODE. At least two queries shall be the same as in Step 1.5; at least one of these has to use more than one word. Use different operators in your queries (at least *, -, and +). Explain results. Prove their correctness. Please compare the expressive power of the IN BOOLEAN MODE and IN NATURAL LANGUAGE MODE. For every query, please report: (i) the query; (ii) the first 20 results with the highest relevance; (iii) your explanations about the correctness and utility of the results; (iv) the comparison the result to the result of the similar query of Step 1.5, if relevant.

1.8 Conclude Your Report

In the conclusion, please state concisely what has been done. Please also inform if the homework was useful for you if you wish to do so.

2 Full-Text Search Using MySQL

Many DBMS support full-text search methods. These methods enable quick search of data in big textual data fields. In difference to MySQL LIKE operator ([1] Sect. 12.8.1) full-text search assumes that a corresponding index is created for the table in the Database, that serves as a dictionary containing the links to the words in the text fields.

To split texts into words, MySQL uses a very simple syntactic analyser. A word is any sequence of symbols – letters, digits, «'» and «_» symbols. A word does not contain blank symbols. Most often, such a sequence contains more than 3 symbols. Any word, that is included into the stop-list, a so-called stop-word, or having less than 4 symbols in length, is ignored. Every correct word is checked against the text and is evaluated, in every particular query, for its significance with respect to this query of the set of texts. For example, a search for the word «UCU» in the UCU News Database will not yield any result as the word is too short (3 symbols). On the other hand, a search for «Catholic» in the UCU News Database will return plenty of results as the word is significant for this texts collection.

MySQL supports full-text indexing and search for MyISAM tables. For performing full-text search in MySQL special FULLTEXT indexes must be used. These indexes could be created for MyISAM tables when these tables are created using the CREATE TABLE statement ([1] Sect. 13.1.20), or added later using the ALTER TABLE statement ([1] Sect. 13.1.9) or CREATE INDEX statement ([1] Sect. 13.1.15). A FULLTEXT index could be created only for the attributes (columns) having values of CHAR, VARCHAR, or TEXT datatypes.

As an example, let us create two **FULLTEXT** indexes for the table `passenger_bio` using the **CREATE TABLE** statement:

```
DROP TABLE IF EXISTS passenger_bio;
CREATE TABLE IF NOT EXISTS passenger_bio
(
    id                smallint NOT NULL auto_increment,
    full_name         varchar (50) NOT NULL,
    bio               text,

    PRIMARY KEY (id),
    FULLTEXT KEY ft1 (full_name, bio),
    FULLTEXT KEY ft2 (bio)
) ENGINE=MyISAM DEFAULT CHARSET=UTF8MB4;
```

In this example, we create the following indexes:

- **ft1** that could be used for full-text search in the fields `full_name` and `bio`
- **ft2** that could be used for full-text search only in the field `bio`, but not in `full_name`

2.1 MATCH ... AGAINST Operator

Full-text search is performed using the MySQL **MATCH (col1,col2,...) AGAINST (expr [search_modifier])** operator ([1] Sect. 12.10) which compares the values of the fields belonging to the attribute list, included in the **FULLTEXT** index, to the words in the text. This function could work in different modes specified in the `[search_modifier]`. As a result, the relevance value is returned which means the degree of similarity between the search phrase (*expr*) and the text in the fields. Relevance is computed differently – depending on the mode `[search_modifier]`.

2.2 Full-Text Search Modes

In MySQL MyISAM tables, full-text search could be performed in

- NATURAL LANGUAGE mode
- NATURAL LANGUAGE mode WITH QUERY EXPANSION
- BOOLEAN MODE

These modes and their specifics are described below in Sect 2.2.1 – 2.2.3.

2.2.1 IN NATURAL LANGUAGE MODE

NATURAL LANGUAGE MODE – is the default search mode in MySQL. A query in **NATURAL LANGUAGE MODE** may look like:

```
SELECT id, bio, MATCH (full_name, bio)
    AGAINST ('president union' IN NATURAL LANGUAGE MODE)
    AS score
FROM passenger_bio WHERE MATCH (full_name,bio)
    AGAINST('president union' IN NATURAL LANGUAGE MODE)
ORDER BY score;
```

In this query, the words 'president union' are searched in the fields of the `full_name` and `bio` of the `passenger_bio` table using **ft1** index.

The output will be automatically sorted by relevance as the sorting condition (**ORDER BY**) is not specified in the **WHERE** clause. The value of relevance in this mode depends on:

- How close is the searched word to the beginning of the text; and
- The proportion of the number of the occurrences of the searched word to the overall number of words in the field

For example, the relevance of the words 'president union' would be higher than 0 if it will be met either in `full_name` or in `bio`. However, if it is met in both `full_name` and `bio` then the relevance will be higher than if it is met 2 times in `bio` only.

Relevance is in fact a measure of the significance of a word in this table (texts collection). It might be useful, therefore, to sort the results of search by this measure to observe which words are more or less significant in carrying the semantics of the collection.

It is good to remember that **IN NATURAL LANGUAGE MODE** the rule of «50% threshold» is applied to search results. Due to this rule, if a word occurs in more than 50 percent of all the fields under **FULLTEXT** search then the results of the search for this word will be returned.

2.2.2 IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION

This mode is a.k.a. **WITH QUERY EXPANSION**. This mode increases the recall of search. However, it is not recommended for big amounts of data as it might return many not very relevant or irrelevant rows. Hence, the precision will be lower.

Full-text search supports query expansion (and in particular, its variant “blind query expansion”). This is generally useful when a search phrase is too short, which often means that the user is relying on implied knowledge that the full-text search engine lacks. For example, a user searching for “Database” may really mean that “MySQL”, “Oracle”, “DB2”, and “RDBMS” all are the phrases that should match “Databases” and should also be returned. This is implied knowledge.

Blind query expansion (also known as automatic relevance feedback) is enabled by adding **WITH QUERY EXPANSION** or **IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION** following the search phrase. It works by performing the search twice, where the search phrase for the second search is the original search phrase concatenated with the few most highly relevant documents from the first search. Thus, if one of these documents contains the word “Databases” and the word “MySQL”, the second search finds the documents that contain the word “MySQL” even if they do not contain the word “Database”. The following example shows this difference:

```
mysql> SELECT * FROM articles
      WHERE MATCH (title,body)
      AGAINST ('Database' IN NATURAL LANGUAGE MODE);
+-----+-----+-----+
| id | title                | body                                |
+-----+-----+-----+
| 1 | MySQL Tutorial       | DBMS stands for Database ...      |
| 5 | MySQL vs. YourSQL    | In the following Database comparison ... |
+-----+-----+-----+

mysql> SELECT * FROM articles
      WHERE MATCH (title,body)
      AGAINST ('Database' WITH QUERY EXPANSION);
+-----+-----+-----+
| id | title                | body                                |
+-----+-----+-----+
| 5 | MySQL vs. YourSQL    | In the following Database comparison ... |
| 1 | MySQL Tutorial       | DBMS stands for Database ...      |
| 3 | Optimizing MySQL     | In this tutorial we show ...      |
| 6 | MySQL Security       | When configured properly, MySQL ... |
| 2 | How To Use MySQL Well | After you went through a ...      |
| 4 | 1001 MySQL Tricks    | 1. Never run mysqld as root. 2. ... |
+-----+-----+-----+
```

2.2.3 IN BOOLEAN MODE

In this mode, the relevance of a word is computed differently wrt the **IN NATURAL LANGUAGE MODE**. It is a conditional measure of correspondence to the given pattern. The offset of the searched pattern from the beginning of the text and the number of found occurrences do not matter. An important particularity of this mode is the possibility of using logical operators

The following logical operators are supported in the **BOOLEAN MODE**:

- “+” – this operator, if it leads or trails a word, states that this word should obligatorily be present in every returned field
- “-” – this operator, if it leads or trails a word, states that this word should obligatorily NOT be present in every returned field

By default (neither + nor - are specified for a word) the word should not obligatorily be present in every returned field. However, if present, the fields that contain it will be more significant, therefore – higher ranked in the query output – similarly to the **IN NATURAL LANGUAGE MODE**.

- “<” or “>” operators are used to decrease (<) or increase (>) the contribution of the word in the relevance that is computed for the row of the fields
- () – round brackets are used to group words in expressions
- ~ – tilde preceding a word is used as the negation operator, which implies negative contribution of this word in the relevance of the row where the word is found. This row will have decreased relevance but will not be excluded as it is in the case of using “-”.
- * – the asterisk serves as the truncation (or wildcard) operator. Unlike the other operators, it is *appended* to the word (prefix) to be affected. Words match if they begin with the word preceding the * operator.
- “” – a phrase that is enclosed within double quotes (") matches only rows that contain the phrase *literally, as it was typed*. The full-text engine splits the phrase into words and performs a search in the FULLTEXT index for the words. Non-word characters need not be matched exactly: Phrase searching requires only that matches contain exactly the same words as the phrase and in the same order. For example, "test phrase" matches "test, phrase". If the phrase contains no words that are in the index, the result is empty. The words might not be in the index because of a combination of factors: if they do not exist in the text, are Stop Words, or are shorter than the minimum length of indexed words.

The following examples demonstrate some search strings that use boolean full-text operators:

- 'apple banana'

Find rows that contain at least one of the two words.

- '+apple +juice'

Find rows that contain both words.

- '+apple macintosh'

Find rows that contain the word “apple”, but rank rows higher if they also contain “macintosh”.

- '+apple -macintosh'

Find rows that contain the word “apple” but not “macintosh”.

- '+apple ~macintosh'

Find rows that contain the word “apple”, but if the row also contains the word “macintosh”, rate it lower than if row does not. This is “softer” than a search for '+apple -macintosh', for which the presence of “macintosh” causes the row not to be returned at all.

- '+apple +(>turnover <strudel)'

Find rows that contain the words “apple” and “turnover”, or “apple” and “strudel” (in any order), but rank “apple turnover” higher than “apple strudel”.

- 'apple*'

Find rows that contain words such as “apple”, “apples”, “applesauce”, or “applet”.

- `"some words"`

Find rows that contain the exact phrase “some words” (for example, rows that contain “some words of wisdom” but not “some noise words”). Note that the " characters that enclose the phrase are operator characters that delimit the phrase. They are not the quotation marks that enclose the search string itself.

This is an example of full-text search IN BOOLEAN MODE:

```
SELECT *,
MATCH (category_title, category_description)
AGAINST ('+Computer +Science -Economy ' IN BOOLEAN MODE) as REL
FROM dl_categories
WHERE MATCH (category_title, category_description) AGAINST ('+Computer +Science
-Economy' IN BOOLEAN MODE)
ORDER BY REL;
```

As a result all the rows will be returned that contain the word “*Computer*” AND the word “*Science*”, AND do NOT contain the word “*Economy*”. The rows will be sorted (ranked) by relevance (REL).

The rule of «50% threshold» **does not apply** to the IN BOOLEAN MODE. Furthermore, this mode could be used for search without having a FULLTEXT index, though substantially slower.

2.3 MySQL Server Configuration Changes

It is good to know that full-text search is normally very finely tuned to ensure the highest possible efficiency of its performance. Any blind changes of the default working mode configuration parameters may result in substantial decrease of performance and lower the quality of results.

- The minimal length of a word that will be indexed is set using MySQL Server configuration variable `--ft-min-word-len`
- The list of the stop words (stop-word list) could be stored in the file with the name specified in MySQL Server configuration variable `--ft-Stop Word-file`. It is necessary to rebuild FULLTEXT indexes if the stop list has been changed.

3 Grading

This homework is graded between 0 and 15. The distribution of the grades per the components of your report is as follows:

- Step 1.1 – 1
- Step 1.2 – 2
- Step 1.3 – 2
- Step 1.4 – 1
- Step 1.5 – 2
- Step 1.6 – 2
- Step 1.7 – 4
- Step 1.8 and overall impression of the report – 1

References

1. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>